

The Production-Grade Vibe Coding Handbook

Everything a developer must verify before AI-generated code touches real users

A zero-shortcuts verification framework for enterprise software built with AI assistance.

Why this handbook exists

AI tools now write a large share of the world's new code. That's not a problem by itself — it's a force multiplier. The problem is what happens when "the AI wrote it and it ran" gets treated as "the AI wrote it and it's *done*." Those are very different claims, and the gap between them is where production incidents, breaches, and outages live.

Three things make AI-generated code structurally different from human-written code, and they are the reason this handbook exists:

- 1. It looks finished, so it gets less scrutiny.**

Reviewers — including the developer who

prompted it — apply less critical attention to code that compiles, runs, and reads cleanly than they would to a colleague's rough draft. Confidence in the output is not correlated with correctness of the output.

2. **It fails silently and specifically.** AI models rarely generate code that crashes on the happy path. They generate code that works for the case you described and quietly omits the authorization check, the rate limit, the input boundary, or the edge case you didn't think to mention. These are *invisible-by-design* defects: they pass casual testing and only surface under adversarial or unusual conditions.
3. **It can fabricate with total confidence.** Models hallucinate package names, API methods, configuration flags, and library behavior that sound completely plausible and do not exist — or exist but behave differently than generated. Unlike a human teammate, the model will not say "I'm not sure"; it will write something syntactically perfect and semantically wrong.

None of this is a reason to avoid AI-assisted development — the productivity gains are real and not going away. It's a reason to treat AI output the way a senior engineer treats *any* code they didn't personally write line-by-line: useful, fast, and unverified until proven otherwise. This handbook is the verification

layer. It assumes you are going to use AI to generate a meaningful share of your application, and it gives you the checklist for every layer of the stack you must independently confirm before that code reaches production — no exceptions for "the AI seemed confident."

The one rule that underlies every section below: if no human can explain why a piece of AI-generated code is correct, secure, and necessary, it is not ready to ship — regardless of how clean it looks or how many tests happen to pass.

Table of contents

Part I — Process

1. The vibe coding risk landscape
2. Prompt engineering & spec-writing for reliable generation
3. The AI code review workflow (human-in-the-loop)
4. Common AI hallucinations & failure patterns (what to specifically check for)

Part II — Application layers

5. UI/UX verification
6. Frontend verification
7. Backend & API design verification
8. Database verification

- 9. Authentication
- 10. Authorization
- 11. API input validation
- 12. Rate limiting & abuse prevention
- 13. Caching
- 14. File storage & uploads

Part III – Security, operations, and trust

- 15. Security hardening (OWASP, secrets, supply chain)
- 16. Logging
- 17. Monitoring & observability
- 18. Testing strategy
- 19. CI/CD pipeline
- 20. Docker & containers
- 21. Cloud deployment & scalability
- 22. Backups & disaster recovery

Part IV – Reach and obligations

- 23. SEO (for web applications)
- 24. Accessibility (WCAG)
- 25. Legal & regulatory compliance

Part V – Shipping

- 26. The final production launch checklist

Appendices

- A. Condensed master checklist (printable)
 - B. Recommended tooling by category
-

Part I — Process

1. The vibe coding risk landscape

"Vibe coding" — describing what you want in natural language and letting an AI agent generate the implementation — has moved from weekend side projects into enterprise production systems faster than most security and engineering organizations have adapted their review processes. A few data points to calibrate how seriously to take this:

- Independent trackers that trace CVEs back to their introducing commit have found AI-attributed vulnerabilities climbing month over month as AI-generated code volume grows.
- Industry surveys consistently report that fewer than half of developers say they *always* review AI-generated code before committing it — while the overwhelming majority use AI tools daily. High usage plus inconsistent review is the exact combination that produces systemic risk.
- The most commonly cited vulnerability patterns in AI-generated code are not exotic: improper input validation, missing or broken object-level authorization, over-permissive cloud IAM roles, and hardcoded credentials. These are precisely the categories this handbook drills into.

- A new supply-chain attack class — **slopsquatting** — exists specifically *because* models hallucinate plausible-sounding package names with predictable regularity, and attackers pre-register those exact names on public registries loaded with malware. This is covered in depth in Section 4.

The core failure mode to internalize: AI-generated code is not "buggier" in the way untested human code is buggy. It is buggy in a *correlated* way — the same blind spots (auth checks, edge cases, security headers, error handling) get skipped across many generations because the model is pattern-matching to the common case described in the prompt, not reasoning about your specific threat model. That correlation is what turns "AI saved me time" into "AI introduced the same hole in twelve endpoints."

What actually works, according to teams that have integrated AI generation without the resulting technical debt or breaches:

- AI-generated code is treated as a first draft from a fast, occasionally-confabulating junior contributor — never as a merge-ready deliverable.
- Human review and automated security/test gates are mandatory at every production boundary, not optional steps that get skipped under deadline pressure.

- Every layer of the stack in Parts II–IV of this handbook has an explicit, named owner who signs off — not "someone probably checked this."
-

2. Prompt engineering & spec-writing for reliable generation

The highest-leverage place to prevent AI coding defects is *before* generation, not after. A vague prompt produces code that is *plausible* for the median interpretation of your request — which is rarely your actual requirement, especially for security-sensitive or business-critical logic.

2.1 Write a spec, not a wish

Treat every non-trivial prompt as a lightweight specification document, not a conversational request. At minimum, include:

- [] **Success criteria** — what "done" looks like, stated as testable conditions, not adjectives. ("Returns 403 if the requesting user does not own the resource" beats "handle permissions properly.")
- [] **Explicit constraints** — language/framework versions, libraries already in use, patterns the codebase follows, things the model must *not* do

(no new dependencies, no schema changes, no changes outside listed files).

- [] **Inputs and context** — the minimum relevant code, schema, or interface the model needs. Don't make it guess at your existing conventions.
- [] **Negative examples** — what should happen on bad input, unauthorized access, empty states, and failure of downstream calls. Models default to writing for the happy path unless told otherwise.
- [] **An output contract** — required file(s), function signatures, error shapes, and what to do when uncertain (e.g., "if a requirement is ambiguous, stop and ask rather than guessing").

2.2 Practical technique notes

- **Keep prompts tight and structured over long and exhaustive.** Model reasoning quality measurably degrades well before hitting technical context limits. A labeled, ~150–300 word spec with headers (**CONTEXT:** , **TASK:** , **CONSTRAINTS:** , **OUTPUT FORMAT:**) consistently outperforms a 1,000-word wall of text — split large tasks into smaller bounded ones instead.
- **Include your real test cases in the prompt**, not just the requirements. Models write code that satisfies the tests you hand them far more reliably than code that satisfies a prose description alone.

- **Use few-shot examples for anything where house style or output shape matters** — two or three examples of the pattern you want, pulled from your own codebase, reliably beats a paragraph of description.
- **State explicitly what the model must verify, not assume.** Models will silently assume a function exists, a field is present, or a package is available unless told to confirm it against the actual codebase/docs first.
- **Version-control your prompts for anything repeated.** If a prompt generates production code more than once, it deserves the same change discipline as the code it produces — review, diffing, and a record of what changed and why.
- **Don't hand-roll chain-of-thought for modern reasoning models.** Current-generation reasoning models already plan internally; forcing an explicit "think step by step" preamble is mostly useful for cheaper, non-reasoning models and can occasionally hurt frontier reasoning models. Spend the saved prompt space on constraints and examples instead.

2.3 Scope every agentic task narrowly

When using an agentic coding tool (one that edits files and runs commands on its own), bound the blast radius explicitly:

- [] Name the exact files/directories the agent is allowed to touch.
 - [] State what it must *not* do (no new dependencies, no infra changes, no migrations) unless that is the explicit task.
 - [] Require it to output a reviewable diff, not a silent multi-file rewrite.
 - [] Maintain a project-level instructions file (e.g., `AGENTS.md` / `CLAUDE.md`) documenting conventions, forbidden patterns, and architecture decisions so every session starts from the same context instead of re-deriving (and re-guessing) it.
-

3. The AI code review workflow (human-in-the-loop)

A workflow, not a vibe, is what separates teams that ship AI-assisted code safely from teams that ship incidents. The model below has four stages with a hard stop between each.

Stage 1 — Intent before generation

- [] A short intent note exists: what is being built, which existing pattern it follows, and what constraints (security, compliance, performance) apply. Five minutes here prevents the single most

common failure mode: code that is technically correct but contextually wrong for this codebase.

Stage 2 — Automated gates (before any human spends review time)

Nothing reaches a human reviewer until it passes, at minimum:

- ☐ Linting and static analysis
- ☐ Type checking (if applicable)
- ☐ Full existing test suite, green
- ☐ Dependency/package existence and vulnerability scan (see Section 4 — this catches hallucinated packages)
- ☐ Secret scanning (no credentials, keys, or tokens in the diff)
- ☐ SAST scan for injection, broken auth, and insecure patterns

Stage 3 — Human review with an AI-specific lens

Standard code review still applies (correctness, readability, performance) — but AI-generated diffs need additional, specific scrutiny that human-authored diffs usually don't:

- ☐ **Does every import/package actually exist and do what the code assumes?** (Section 4)

- ☐ **Does every object-access endpoint check ownership, not just authentication?** (Sections 7, 10 — this is the single most common AI-generated vulnerability class)
- ☐ **Is there an authorization check on every new function, not just the ones the prompt mentioned?**
- ☐ **Are error paths, empty states, and invalid input handled, or only the happy path?**
- ☐ **Does the business logic match the actual written requirement,** not a plausible-sounding nearby interpretation of it?
- ☐ **Is this the right architectural approach for this system,** or a generic pattern that doesn't fit how the rest of the codebase is structured?
- ☐ **Disclose the AI's role in the PR description** — what was generated vs. hand-written, and what prompt/spec was used. This is not bureaucracy; it tells the next reviewer (including future-you) where to point extra scrutiny.

Stage 4 — Merge and curate

- ☐ Post-merge, capture what worked (and what was rejected and why) in a living team prompt/pattern playbook. Calibrating prompts over time is how review burden goes down without review rigor going down with it.

Splitting the work between AI tools and humans

AI code-review tooling is genuinely good at the mechanical half of review — style, obvious bugs, known-vulnerability patterns, missing tests — and using it to clear that layer automatically is a legitimate efficiency gain. It is measurably *not* good at the half that matters most for AI-authored code specifically: business-logic correctness, whether the architecture fits the system, and whether the implementation matches the actual intent behind the request. Structure your review process so AI tools own the first list and a named human owns the second — and never let "the AI reviewer approved it" substitute for the human pass on architecture and intent.

4. Common AI hallucinations & failure patterns

This is the checklist of *specifically what to verify* because models get it wrong in predictable, recurring ways. Treat each of these as a mandatory check on AI-generated diffs, not a rare edge case.

4.1 Package & dependency hallucination ("slopsquatting")

Models routinely invent plausible-sounding package, library, or SDK names that don't exist — by blending two real package names, generating a typo-adjacent variant, or fabricating one outright when no real package fits the request. Independent research measured this happening in roughly one out of every five AI code-generation samples across tested models, with the exact same fake names recurring across repeated runs of the same prompt — which is exactly what makes them exploitable.

This stopped being theoretical: attackers now monitor AI output for commonly hallucinated names, register those exact names on public registries (npm, PyPI, etc.) loaded with credential-stealing install scripts, and wait. Documented incidents include a hallucinated package accumulating tens of thousands of downloads from an experiment alone, and a fabricated package propagating through hundreds of repositories via AI-generated automation with no human in the loop at all.

Mandatory checks:

- [] Every new dependency is verified to exist on the real registry *before* install — not assumed from the AI's output.
- [] Verify the package was registered before your project started, has a credible maintainer history, and matches the ecosystem you're actually using

(watch for cross-language confusion, e.g., a Python-sounding name resolved against npm).

- [] Use an allow-list or automated registry-verification tool in CI that blocks installs of anything not already vetted (Section 19).
- [] Generate and retain a Software Bill of Materials (SBOM) for every build so a later-discovered hallucinated/malicious package can be traced to every affected release.
- [] Run installs for unfamiliar packages in an isolated, ephemeral environment first, never directly in a privileged CI runner or developer machine with live credentials.

4.2 Fabricated APIs, methods, and config flags

Models confidently generate calls to methods that don't exist on a given library/version, reference config options that were removed or renamed, and invent endpoint shapes for third-party APIs. The code often still *compiles* (especially in dynamically typed languages) and only fails at runtime, sometimes only on a specific input.

- [] Every call into a third-party SDK or internal service is checked against the actual installed version's documentation — not the model's memory of an unspecified version.

- [] Anything touching an external API is validated against a real request in a sandbox/staging environment, not just read for plausibility.

4.3 Authorization gaps masquerading as working features

The most common security defect in AI-generated backend code: an endpoint correctly checks that *a* user is logged in, but never checks that *this* user is allowed to access *this specific resource*. The feature works perfectly in every manual test a developer runs (because they test with their own data) and is silently broken for every other user. See Section 10 for the full authorization checklist — but flag this here because it is the single highest-frequency, highest-impact AI hallucination-adjacent defect in production incidents.

4.4 Confidently incorrect business logic

Code that is syntactically perfect, passes the tests the model itself wrote, and implements the *wrong* rule — an off-by-one in a billing calculation, an inverted condition in a discount eligibility check, a rounding rule that doesn't match the spec. Models do not flag uncertainty here; they commit to an interpretation and write it fluently.

- [] Business-critical logic is verified against the written requirement by a human who knows the

requirement, not just against tests the same AI session generated.

- [] Tests for critical logic are written or reviewed independently of the implementation generation, ideally by a different prompt/session or a human, to avoid the model grading its own homework.

4.5 Plausible-looking security anti-patterns

Patterns that look like security best practice but aren't: rolling your own crypto instead of a vetted library, comparing secrets with `==` instead of constant-time comparison, "validating" input with a regex that misses encoded payloads, or storing a token in `localStorage` because it's the most common pattern in public training data — not because it's safe for your threat model.

- [] Anything touching authentication, cryptography, or secret comparison is checked against an actual current security cheat sheet (e.g., OWASP Cheat Sheet Series), not approved on the basis of "it looks like what tutorials do."

4.6 Stale or deprecated patterns

Models trained on a snapshot of public code reproduce whatever was *common*, which is not the same as *current*. Deprecated auth libraries, outdated

dependency-pinning practices, and abandoned security patterns show up regularly in generated code.

- [] Cross-check generated code against your framework's current official docs, not the model's training-data-era defaults — especially for anything related to auth, crypto, or recently-changed APIs.

4.7 The meta-check

Before merging any AI-generated diff, a human should be able to answer, specifically:

- [] "What does this code do if the input is empty, malformed, or hostile?"
- [] "What does this code do if the user is authenticated but shouldn't have access to *this* object?"
- [] "Do I know that every import in this diff is a real, vetted package?"
- [] "If I had written this myself, would I be confident defending it in an incident postmortem?"

If the honest answer to any of these is "I'm not sure," the diff is not ready — regardless of how clean it reads.

Part II — Application layers

5. UI/UX verification

AI-generated UI tends to be visually plausible and functionally incomplete — it nails the default state and skips the states that make an interface trustworthy in production.

5.1 State coverage

- [] **Loading states** exist for every async operation (skeletons/spinners), not just a blank screen until data arrives.
- [] **Empty states** are designed, not just "no data" — first-time users and zero-result searches need explicit guidance, not a blank table.
- [] **Error states** are specific and actionable ("Couldn't save — check your connection and retry" beats a generic "Something went wrong"), and never leak stack traces or internal identifiers to the user.
- [] **Partial-failure states** are handled when a screen depends on multiple data sources — one failing shouldn't blank the whole page if the others succeeded.
- [] **Offline / slow-network behavior** is verified, not assumed — throttle the network in devtools and

confirm the UI degrades gracefully rather than hanging or duplicating submissions.

5.2 Form and input UX

- [] Validation errors are inline, specific, and appear without losing the user's other entered data.
- [] Destructive actions (delete, cancel subscription, remove member) require confirmation and clearly state what will happen — AI-generated "delete" buttons routinely fire immediately with no confirmation step.
- [] Double-submit is prevented (disable the button / debounce) — generated forms frequently allow a fast double-click to fire two requests.
- [] Field-level constraints in the UI (max length, allowed characters, format) match the backend's actual validation rules — mismatches here create confusing "it looked valid but the server rejected it" experiences.

5.3 Responsive and cross-device behavior

- [] Verified at real breakpoints on real devices/emulators, not just by shrinking a desktop browser window — AI-generated CSS frequently "works" at the one viewport it was eyeballed at and breaks at common phone widths.

- [] Touch targets meet a minimum size (effectively ~24×24 CSS px per WCAG 2.2, commonly implemented as 44×44px for comfortable use) – generated UI often copies desktop-density spacing onto mobile.
- [] Tested with real content lengths (long names, long error messages, many list items), not just the placeholder text used during generation.

5.4 Visual and interaction consistency

- [] A single design system/token set (spacing, color, type scale) is enforced – AI generation across multiple sessions/prompts tends to introduce subtly inconsistent spacing, button styles, and color usage that compounds across a codebase.
 - [] Focus states, hover states, and active states are present and visible for every interactive element (also an accessibility requirement – see Section 24).
 - [] Animations and transitions respect `prefers-reduced-motion`.
-

6. Frontend verification

6.1 Correctness and rendering

- [] No layout-shifting content: images, embeds, and ads have reserved dimensions (explicit `width / height` or `aspect-ratio`) so they don't push content around after load — this is both a UX and a Core Web Vitals (CLS) issue (Section 23).
- [] Client-side state management doesn't silently drop or duplicate data on race conditions (rapid navigation, concurrent requests, stale closures in async callbacks) — a frequent AI-generated bug class in hooks/effects-heavy code.
- [] Memory leaks checked: event listeners, intervals, and subscriptions are cleaned up on unmount; long-running SPA sessions don't accumulate detached DOM nodes or growing memory.

6.2 Performance

- [] Bundle size audited; unused dependencies pulled in by generated code are removed (models frequently add a heavyweight library for something a few lines of native code would do).
- [] Code-splitting/lazy-loading applied to routes and heavy components, not a single monolithic bundle.
- [] Long-running JavaScript tasks (>50ms) on the main thread are identified and broken up — this is the direct cause of failing INP scores (Section 23).

- [] Images use modern formats (WebP/AVIF) with responsive `srcset`, and the largest above-the-fold image (the likely LCP element) is never lazy-loaded.

6.3 Client-side security

- [] No secrets, API keys, or internal URLs are present in client-bundled code — anything shipped to the browser is public by definition, and AI-generated code regularly puts a "backend" key directly into frontend config because it solves the immediate task fastest.
- [] User-generated content rendered as HTML is sanitized, not injected via `dangerouslySetInnerHTML` / `innerHTML` - equivalents without a sanitizer — a common AI-generated XSS vector (OWASP A05:2025 — Injection).
- [] Sensitive tokens are not stored in `localStorage` / `sessionStorage` unless you've explicitly accepted the XSS-exfiltration tradeoff; prefer `httpOnly`, `secure`, `SameSite` cookies for session tokens (Section 9).
- [] Content Security Policy (CSP) headers are set and actually restrict script sources — not left at a generated placeholder or absent entirely.

6.4 Cross-browser and degradation

- [] Verified on the actual browser matrix your users use (not just the one the AI tool's preview ran in) – Safari- and older-Android-specific CSS/JS bugs are a common gap.
 - [] Critical functionality has a fallback if JavaScript fails to load or a third-party script is blocked (ad blockers, corporate firewalls).
-

7. Backend & API design verification

7.1 Object-level authorization on every endpoint

This is the single highest-frequency vulnerability class in both AI-generated code and APIs generally (it has held the #1 spot on the OWASP API Security Top 10 since 2019). The pattern: an endpoint correctly confirms the requester is *authenticated*, then fetches an object by an ID taken from the request – without confirming the authenticated user actually *owns or has rights to* that specific object.

- [] For every endpoint that accepts an object ID (`/orders/{id}` , `/documents/{id}` , GraphQL resolvers included), write a test with **two real accounts**: create a resource as Account A, then attempt to read/modify/delete it as Account B. If

B succeeds, this is a Broken Object Level Authorization (BOLA) vulnerability.

- [] Ownership/permission checks happen **server-side, on every request**, not inferred from a client-supplied role field, a hidden form field, or "the UI doesn't show that button to this role."
- [] Bulk/list endpoints filter by ownership/permission at the query level — not fetched broadly and filtered in application code after the fact (which leaks existence/count information and risks a missed filter path).

7.2 Object *property*-level authorization

A related, distinct issue: a user may legitimately access an object but should not be able to read or write *every field* on it (e.g., a `role` or `isAdmin` flag, internal pricing notes, another user's private fields nested in a shared resource).

- [] Mass-assignment is disabled by default — incoming request bodies are mapped to an explicit allow-list of fields, never deserialized directly onto a full database model.
- [] Response serialization uses an explicit allow-list of fields per audience (admin view vs. public view), not "return the whole object and let the frontend not display the sensitive parts."

7.3 Function-level authorization

- [] Admin/privileged routes check role/permission server-side on every call, not only at the UI layer or only on the "main" entry point to a feature (a frequent gap: the primary admin page checks the role, but an underlying API route it calls does not, and is reachable directly).
- [] Permission checks are centralized in one reusable layer (middleware/decorator/policy engine), not re-implemented ad hoc per endpoint – ad hoc checks are exactly where AI-generated inconsistency creeps in across a large surface.

7.4 Business-flow abuse

Some of the most damaging API issues aren't bugs at all – the API does exactly what it was built to do, and that's the problem (e.g., no limit on how many times a "create discount code" or "send invite" endpoint can be called, enabling abuse at scale).

- [] Sensitive flows (signup, checkout, invites, password reset, coupon redemption) have abuse limits independent of generic rate limiting – CAPTCHA, velocity checks, or step-up verification where appropriate.

7.5 Server-side request forgery (SSRF)

- [] Any feature that fetches a URL supplied by a user (webhooks, "import from URL," link previews, image-by-URL) validates and restricts the destination — block requests to internal/private IP ranges and cloud metadata endpoints (`169.254.169.254` and equivalents). AI-generated "fetch this URL and return the result" code very commonly omits this entirely.

7.6 Idempotency and concurrency

- [] State-changing endpoints that can plausibly be retried (payments, order creation) support idempotency keys so a network retry doesn't double-charge or double-create.
- [] Concurrent updates to the same resource use optimistic locking/versioning or transactions — generated CRUD code frequently does a naive read-modify-write with no protection against lost updates.

7.7 API contract discipline

- [] An API spec (OpenAPI/GraphQL schema) exists and is the source of truth — not reverse-engineered from whatever the AI happened to generate.
- [] Breaking changes are versioned; old API inventory is tracked and deprecated deliberately (an out-of-date, undocumented "zombie" endpoint

that nobody remembers exists is a top-10 API risk in its own right — Improper Inventory Management).

- [] Consistent error response shape across all endpoints, with no internal exception messages, stack traces, or database errors ever returned to the client.

8. Database verification

8.1 Schema and data integrity

- [] Every foreign key relationship that should exist has an actual foreign key constraint — not just application-level "we always set this correctly," which AI-generated CRUD code routinely assumes.
- [] `NOT NULL`, `UNIQUE`, and `CHECK` constraints are enforced at the database layer for invariants that matter (an email column, a non-negative balance, a valid enum), not only validated in application code that a future code path could bypass.
- [] Cascade behavior on delete/update (`CASCADE`, `RESTRICT`, `SET NULL`) is deliberately chosen per relationship, not left at whatever default the generated migration happened to pick — an

unreviewed `ON DELETE CASCADE` on the wrong relationship is a quiet way to mass-delete data.

- [] Money/currency fields use a precise decimal type, never floating point.

8.2 Query correctness and performance

- [] No string-concatenated SQL anywhere — parameterized queries / an ORM's query builder only. This is the textbook SQL injection vector and still shows up in AI-generated "quick script" code that bypasses the ORM for a one-off query.
- [] N+1 query patterns are checked for explicitly — a generated list view that loops and queries per item is a very common AI output and degrades silently until traffic grows.
- [] Indexes exist for every column used in a `WHERE`, `JOIN`, or `ORDER BY` on a table expected to grow — verified against an actual query plan (`EXPLAIN`), not assumed.
- [] Pagination is implemented for every list endpoint before it ships, not retrofitted after the first table scan timeout in production. Cursor-based pagination is preferred over offset-based for large/changing datasets.

8.3 Migrations

- [] Migrations are reviewed for backward compatibility with the *currently deployed* application version — a migration that drops/renames a column the live app still reads will break mid-deploy.
- [] Destructive migrations (drop column/table) ship as a separate, later step after the application code has stopped using the field — never in the same deploy that removes the application's usage.
- [] Large migrations on production-scale tables are checked for lock behavior; long-held table locks during a migration are a common cause of "the deploy looked fine but the site went down for two minutes."

8.4 Multi-tenancy and data isolation

- [] If the application is multi-tenant, isolation is enforced at the lowest practical layer (row-level security, a mandatory tenant-scoped query layer, or separate schemas/databases per tenant) — not solely by "every query includes a `WHERE tenant_id = ?`" that a future AI-generated query could simply omit.
 - [] Database credentials used by the application have least-privilege grants (no superuser/admin connection string for routine application queries).
-

9. Authentication

9.1 Choosing the mechanism

As of 2026 the landscape has converged on a few well-understood building blocks rather than a single "right answer" — pick deliberately:

Mechanism	Best for	Key risk if misused
Session cookies (server-side state)	Traditional server-rendered/monolithic apps	Session fixation, CSRF if not paired with protections
JWT / stateless tokens	APIs, SPAs, microservices	Hard to revoke before expiry; never put authorization decisions inside an unverified claim
OAuth 2.1 + OpenID Connect	Third-party/social login, delegated access, SSO	Misconfigured redirect URIs, missing PKCE

Mechanism	Best for	Key risk if misused
		on public clients
Passkeys / WebAuthn (FIDO2)	Consumer auth, phishing resistance	Recovery-flow and account-linking edge cases
SAML 2.0	Enterprise B2B SSO	Legacy, XML-based; still the default for many enterprise IdPs

- [] **Passwords, if used at all, are hashed with a modern adaptive algorithm** (bcrypt, scrypt, or Argon2) with appropriate cost parameters — never MD5/SHA-1/plain SHA-256, and never a roll-your-own scheme.
- [] **New user-facing auth should default toward passkeys/WebAuthn or a managed passwordless flow** where feasible; if passwords remain (legacy, fallback), pair them with mandatory or strongly encouraged MFA.
- [] Multi-factor authentication is available, and required for privileged/admin accounts at

minimum.

9.2 Session and token hygiene

- [] Access tokens are short-lived (commonly ~15 minutes–1 hour); refresh tokens are longer-lived, opaque, and revocable server-side — generated code frequently issues a single long-lived token because it's simpler, which removes your ability to revoke a compromised session quickly.
- [] Logout actually invalidates the session/token server-side (revocation list, session store deletion) — not just deleting the client-side cookie/localStorage entry, which a generated "logout" implementation will do by default and leaves the token valid if stolen.
- [] Session cookies are `httpOnly`, `Secure`, and `SameSite=Lax / Strict` as appropriate.
- [] Session ID is rotated on privilege change (login, password change, role escalation) to prevent session fixation.
- [] All JWT claims are verified on every request: signature, issuer, audience, expiry, and key ID — not just "the token is present." Confirm the signing algorithm is pinned server-side (never trust an `alg` value the client/token claims to use; reject `none`).

9.3 Account security flows

- [] Login responses do not reveal whether the failure was a bad username vs. a bad password (prevents account enumeration).
- [] Rate limiting and progressive lockout/backoff are applied to login, password reset, and MFA verification endpoints specifically – these are the highest-value brute-force/credential-stuffing targets and are frequently left unprotected in generated auth scaffolding (Section 12).
- [] Password reset tokens are single-use, short-lived, and invalidate all other outstanding reset tokens for that account once used.
- [] Email/phone verification flows can't be used to enumerate existing accounts (don't reveal "this email is already registered" in ways that leak account existence unnecessarily, depending on your threat model).

9.4 If delegating to a third-party identity provider

- [] Redirect URIs are an exact allow-list, not a wildcard/prefix match.
- [] PKCE is used for any public client (mobile, SPA).
- [] The `state` parameter is used and validated to prevent CSRF on the OAuth callback.

- [] ID token claims are verified the same way JWTs are verified above — don't trust the access token as a proxy for identity.
-

10. Authorization

Authentication answers "who are you?" Authorization answers "are you allowed to do *this specific thing*, to *this specific resource*, right now?" — and it has to be evaluated on every request, not just at login. This is the layer where AI-generated code most reliably falls short, because the model satisfies the prompt's literal ask ("let users view their orders") without independently reasoning about everyone who *shouldn't* be able to.

10.1 Model selection

- [] **RBAC (role-based)** for straightforward permission sets — clear until you have many fine-grained resource-specific rules, at which point roles multiply uncontrollably.
- [] **ABAC/policy-based (attribute-based)** — e.g., a centralized policy engine evaluating user attributes, resource attributes, and context — for anything with conditional, resource-specific, or relationship-based rules ("a manager can approve

their own team's requests but not another team's").

- [] Whichever model is chosen, **permission logic lives in one centralized, testable layer** — not duplicated inline across dozens of AI-generated endpoints, where it will inevitably drift out of sync.

10.2 Enforcement discipline

- [] **Default deny.** Every new endpoint starts with no access and an explicit grant is added — never "open by default, restrict later," which is exactly the gap that ships when a deadline arrives before "later" does.
- [] Authorization checks happen **server-side only**. Anything enforced purely in client UI (hiding a button, disabling a menu item) is a UX nicety, not a security control, and must never be the only check.
- [] Every new code path that returns or mutates data is checked against this question: *"Have I confirmed this specific user can act on this specific resource, right here, regardless of how they arrived at this code path?"*
- [] Authorization failures are logged with enough context (who, what resource, what action) to detect probing/enumeration attempts (Section 16).

10.3 Principle of least privilege, end to end

- [] Service-to-service calls and background jobs run with scoped credentials for exactly what that job needs — not the same broad credential used everywhere because it was the one already in context during generation.
- [] Database roles, cloud IAM roles, and API keys are scoped per service/function. Over-permissive IAM role assignment is one of the most commonly cited issues in AI-generated infrastructure code specifically — a model asked to "give this service access to the bucket" will often grant broad account-level access because it's the fastest way to make the immediate task work.
- [] Access reviews happen on a schedule — permissions accumulate and are rarely proactively removed.

10.4 AI agents and non-human identities

If any part of your system lets an AI agent act on a user's behalf (calling internal APIs, executing workflows):

- [] The agent has its own scoped identity/credential — never simply handed the user's session token.

- [] Its permissions are constrained explicitly (e.g., "read calendar" but not "delete calendar") rather than inheriting the full scope of whatever it's connected to.
 - [] Every agent action is logged with both the agent's identity and the human it's acting on behalf of.
 - [] There is an immediate, working revocation path — don't rely on a token simply expiring; assume you'll need to cut access off mid-session.
-

11. API input validation

- [] **Validate, don't sanitize-and-hope.** Reject malformed input with a clear error rather than attempting to "clean" it and proceed — silent sanitization is a frequent source of logic bugs and bypass vectors.
- [] Validation happens **server-side on every request**, even for fields the client-side form already validates — the client-side check is a UX convenience, not a security boundary, and is trivially bypassed by anyone calling the API directly.
- [] Use schema-based validation (e.g., a JSON Schema/Pydantic/Zod-style validator) tied to your

API contract, not scattered ad hoc `if` checks that drift from the documented contract over time.

- [] Enforce types, formats, length limits, and allowed value sets — not just "is this field present." A string field with no length cap is an open invitation for storage abuse and downstream rendering/processing issues.
 - [] Validate **structure before content**: reject unexpected fields, deeply nested payloads beyond a sane depth, and oversized request bodies before doing any business logic on them (this also mitigates resource-exhaustion/DoS vectors).
 - [] File upload fields are validated by actual content/magic-bytes inspection, not by trusting the client-supplied filename extension or `Content-Type` header (Section 14).
 - [] Numeric inputs that map to money, quantities, or limits are checked for sane bounds (no negative quantities, no absurdly large values) — a missing upper bound on a generated "quantity" field is a common path to abuse or integer-overflow-adjacent bugs.
 - [] Validation error responses describe *what* was invalid without echoing back unsanitized user input into an HTML or log context (reflected-XSS and log-injection prevention).
-

12. Rate limiting & abuse prevention

- [] **Every public endpoint has a rate limit**, even ones that "shouldn't" need one — AI-generated scaffolding routinely ships with zero rate limiting because the prompt didn't ask for it and the happy-path demo doesn't need it. Unrestricted Resource Consumption is a top-10 API risk in its own right.
- [] Limits are tiered appropriately: stricter on authentication, password reset, and signup endpoints (brute-force/credential-stuffing targets) than on routine read endpoints.
- [] Limiting is enforced **server-side**, keyed on a meaningful identity (authenticated user ID, API key) in addition to IP — IP-only limiting is trivially bypassed with rotating IPs/proxies.
- [] Choose an algorithm deliberately: token bucket or sliding-window log/counter for smooth, burst-tolerant limiting; fixed-window only where its boundary-burst weakness is acceptable.
- [] Rate-limited responses return a proper 429 with a `Retry-After` header, not a generic error or a silent drop.
- [] Pagination and result-size caps are enforced on list endpoints regardless of rate limiting — an attacker (or a buggy client) requesting ?

`limit=999999` shouldn't be able to force a massive query.

- [] Expensive operations (search, export, report generation, anything calling a paid third-party API) have their own tighter limits and, where appropriate, are pushed to an async job queue rather than a synchronous request.
 - [] Rate limit state lives in a shared store (e.g., Redis) in any multi-instance deployment — in-memory counters per server instance silently stop working the moment you scale past one process.
-

13. Caching

- [] **Cache invalidation is explicit, not assumed.**
Every cache write has a clearly defined trigger for when it gets invalidated/updated — "we'll just set a short TTL" is a valid strategy only when you've deliberately decided staleness for that duration is acceptable, not a default to fall back on because invalidation logic wasn't generated.
- [] **No sensitive or per-user data is cached at a shared layer** (CDN, shared application cache) keyed only on a URL that doesn't vary per user — this is a direct path to serving User A's data to User B. Verify cache keys include

user/session/tenant context wherever the response is personalized.

- [] HTTP cache headers (`Cache-Control` , `ETag`) are set deliberately per response type — generated code often either omits them (forcing every request to origin) or copies a permissive default onto an endpoint that returns private data.
 - [] Cache stampede protection exists for hot keys (locking, request coalescing, or staggered expiry) so a popular cache entry expiring under load doesn't send a thundering herd of requests to the database simultaneously.
 - [] A documented cache-clearing/kill-switch path exists for incident response — when bad data gets cached, you need a fast way to purge it, not a redeploy-and-pray.
 - [] Application-level caches (e.g., Redis) have eviction policies and memory limits configured deliberately, not left at framework defaults that could allow unbounded growth.
-

14. File storage & uploads

14.1 Upload handling

- [] File type is verified by inspecting actual content (magic bytes/MIME sniffing), never trusted from

the client-supplied extension or `Content-Type` header alone.

- ☐ Maximum file size is enforced server-side (and ideally at the proxy/load-balancer layer too) before the full file is read into application memory.
- ☐ Uploaded files are scanned for malware where the threat model warrants it (any file later served to other users or opened by staff).
- ☐ Uploaded filenames are never used directly as storage keys or in filesystem paths — generate your own key server-side to eliminate path traversal and overwrite risks from a crafted filename.
- ☐ Images/documents that will be processed (resized, parsed, rendered) are handled with libraries kept current — image/document parsers are a recurring source of CVEs.

14.2 Object storage configuration (S3 and equivalents)

- ☐ Buckets are private by default, with public access blocked at the account level — public access is something you explicitly opt into per-resource via signed URLs or a CDN, never the bucket's baseline state.
- ☐ If a bucket genuinely needs to serve public content, do it through a CDN with origin access

control (so the bucket itself stays private and the CDN is the only path to it) rather than making the bucket itself public.

- [] Legacy ACLs are disabled in favor of bucket policies/IAM — ACLs are a frequent, hard-to-audit source of accidental exposure.
- [] Server-side encryption is enabled by default for data at rest; HTTPS-only access is enforced via a deny-on-insecure-transport policy.
- [] IAM policies grant the minimum specific actions needed (no wildcard `*` actions on production storage).

14.3 Pre-signed / temporary access URLs

- [] Expiry is as short as the use case allows (minutes, not days) — a generated implementation will often default to the maximum allowed expiry because it "just works" during testing.
- [] The object key/path is generated and controlled **server-side** — never accept a client-supplied path to sign, which can allow overwriting or accessing unintended objects.
- [] URL generation is restricted to a dedicated, minimally-scoped signing role/identity, not the application's broad general-purpose credential.
- [] Where the data is sensitive, add IP restrictions or single-use semantics on top of expiry, since a

presigned URL is effectively a bearer token — anyone who obtains it before it expires can use it.

- [] Bucket/object names don't follow a guessable pattern, and any bucket that is deprovisioned is fully removed (not left as a danglely-referenced name another account could later claim).

14.4 Lifecycle and cost

- [] Lifecycle policies move/expire old objects deliberately (especially temporary upload artifacts, logs, and old backups) — unmanaged storage growth is a quiet, compounding cost and data-retention-policy violation risk.
 - [] Versioning is enabled where accidental overwrite/delete protection matters, paired with a lifecycle rule to clean up old versions so costs don't grow unbounded.
-

Part III — Security, operations, and trust

15. Security hardening

15.1 OWASP Top 10:2025 — verify against every category

The OWASP Top 10 is the industry-standard checklist for web application risk. The 2025 edition reshuffled categories to reflect modern attack patterns — supply chain and configuration risk moved up sharply. Walk every one of these against your AI-generated codebase specifically; don't assume coverage because a generic security scan ran once.

#	Category	What to verify
A01	Broken Access Control	Every object/function-level check from Sections 7 & 10 is in place. (SSRF is now folded into this category — see 7.5.)
A02	Security Misconfiguration	No default credentials, debug modes, or verbose error pages in production; cloud services and APIs are configured per vendor hardening guides, not left at defaults.
A03	Software Supply Chain Failures	Dependencies are pinned, vetted, and scanned (Section 15.3); no hallucinated packages (Section 4.1);

#	Category	What to verify
		CI/CD build pipeline integrity is verified (Section 19).
A04	Cryptographic Failures	TLS enforced everywhere; sensitive data encrypted at rest; no home-grown crypto; secrets never logged or hardcoded (Section 15.2).
A05	Injection	Parameterized queries only (Section 8.2); output encoding for any user content rendered as HTML/SQL/shell/LDAP.
A06	Insecure Design	Threat-modeled at the design stage for sensitive flows, not retrofitted after a generated implementation already exists.
A07	Authentication Failures	MFA available, brute-force protections in

#	Category	What to verify
		place, session handling correct (Section 9).
A08	Software or Data Integrity Failures	Build artifacts and CI/CD inputs are verified/signed; deserialization only targets known, expected types.
A09	Security Logging & Alerting Failures	Security-relevant events are logged and actually routed to active alerting, not just written to a file nobody watches (Section 16).
A10	Mishandling of Exceptional Conditions	Errors, timeouts, and unexpected input fail closed/safely, not open – verify what happens when a downstream dependency is slow, errors, or returns malformed data.

15.2 Secrets management

- [] **No secrets in source control, ever** – not in code, not in `.env` files committed "temporarily," not in

CI/CD YAML. Use a pre-commit secret scanner (e.g., gitleaks-class tooling) plus repository-wide scanning, since AI-assisted commits make it easy to paste a working credential into a "just to get it running" snippet that gets committed.

- [] Secrets live in a dedicated secrets manager/vault (cloud-native secrets manager, HashiCorp Vault, or equivalent) — injected at runtime, never baked into a Docker image layer or build artifact.
- [] Secrets are never printed to console output, logs, or shell history during CI/CD execution.
- [] Credentials are scoped per-service/per-pipeline with least privilege; a leaked credential for one service should not grant access to unrelated systems.
- [] Rotation is automated and scheduled, not "we'll rotate it if something happens" — and rotation actually works (test it before you need it for real).

15.3 Software supply chain integrity

- [] Every dependency is pinned to a specific version (lockfiles committed), with automated scanning for known CVEs in both new and existing dependencies on every build.
- [] An SBOM is generated for every release and retained — this is what lets you answer "are we

affected?" in minutes instead of days the next time a dependency-level vulnerability is disclosed.

- [] Build provenance is tracked at least to the level of "we can prove what source produced this artifact and that the build environment wasn't tampered with" (the SLSA framework's levels are a useful maturity ladder: provenance → signed provenance on protected infrastructure → isolated, tamper-resistant builds).
- [] New dependencies go through an actual approval step (even a lightweight one) checking maintenance activity, license, and known-vulnerability history before being added — not added ad hoc because an AI suggested it mid-task.
- [] If AI agents have any ability to install dependencies or trigger builds autonomously, they are treated as a governed identity with logged, auditable actions — not an ungoverned shortcut around your normal dependency-approval process.

15.4 Security headers and transport

- [] HTTPS enforced everywhere (HSTS enabled), with no mixed-content paths.
- [] Security headers set deliberately: Content-Security-Policy, X-Content-Type-Options:

`nosniff`, `X-Frame-Options` /`frame-ancestors`
CSP directive, `Referrer-Policy`.

- [] CORS is configured with an explicit allow-list of origins — never a wildcard `*` on any endpoint that handles authenticated requests or cookies.
- [] CSRF protection is in place for any cookie-authenticated state-changing request.

15.5 Independent verification

- [] Automated SAST/dependency/secret scanning runs on every PR, not just periodically.
 - [] A real penetration test or, at minimum, a focused manual security review happens before launch and after any major architecture change — automated scanning catches known patterns; it does not catch business-logic authorization flaws, which is exactly the category AI-generated code is most prone to.
-

16. Logging

- [] **Structured logging from day one** (JSON or equivalent key-value format) — not raw string interpolation that becomes unparseable the moment you need to search across millions of log lines.

- [] **Security-relevant events are explicitly logged:** authentication successes/failures, authorization denials, password/MFA changes, admin actions, and access to sensitive resources — with enough context (who, what, when, from where) to reconstruct an incident timeline.
- [] **No sensitive data in logs** — passwords, tokens, full credit card numbers, and other secrets/PII are never logged in plaintext, including in error/stack-trace logging that might otherwise capture an entire request object. Add explicit redaction for known sensitive fields.
- [] Logs are correlated across services with a request/trace ID that's generated at the edge and propagated through every downstream call — without this, debugging a multi-service request is guesswork.
- [] Log levels are used meaningfully (debug/info/warn/error) so production log volume is filterable and noisy debug output doesn't drown out real signals.
- [] Logs are shipped to a centralized, access-controlled store with defined retention — not left only on individual container/instance disks that disappear the moment that instance is replaced.
- [] Logging failures themselves fail safe — a logging pipeline outage should degrade observability, not take down the application.

17. Monitoring & observability

- [] **The three pillars are all present:** metrics (system and business-level), structured logs, and distributed traces — logging alone is not observability for anything beyond a single-process app.
- [] Golden signals are tracked for every service: latency, traffic, error rate, and saturation (resource usage) — at minimum.
- [] **Alerts are tied to user-facing impact**, not just infrastructure trivia — alert on elevated error rate, latency SLO breaches, and failed authentication spikes; don't bury the team in noise from every CPU blip, or real alerts will be ignored along with the noise.
- [] On-call has a clear, tested runbook for the alerts that matter, and alerts route to a real notification channel (paging, not just a dashboard nobody is watching).
- [] Uptime/synthetic checks exist for critical user flows (login, checkout, core API), independent of internal metrics, so you find out about an outage from your own monitoring before a user tells you on social media.
- [] Dashboards exist for both engineering health (errors, latency, saturation) and business health

(signups, conversion, key funnel steps) – the second category is what tells you a "successfully deployed, zero errors" release nonetheless broke a business-critical flow.

- [] Distributed tracing (e.g., OpenTelemetry-based) is wired through service boundaries before you need it for an incident, not added reactively after the first multi-service outage you couldn't diagnose.
-

18. Testing strategy

A passing test suite generated alongside the implementation by the same AI session tells you the implementation is internally consistent with itself – it does not tell you the implementation is correct. Tests must be reviewed and, for anything critical, written or substantially verified independently.

18.1 The pyramid, applied deliberately

- [] **Unit tests** cover business logic and edge cases – explicitly including the negative cases the original prompt didn't mention (invalid input, unauthorized access, boundary values, empty collections).
- [] **Integration tests** cover real interactions between your code and its dependencies

(database, cache, external API) — not everything mocked into a sealed bubble that only proves your mocks are self-consistent.

- [] **End-to-end tests** cover the critical user journeys (signup, checkout, core workflow) through the real UI/API layer, run against an environment that resembles production.
- [] Test pyramid shape is intentional: many fast unit tests, a meaningful layer of integration tests, a small set of high-value E2E tests — not the inverse (a few slow E2E tests standing in for proper unit coverage), which is a common shape when tests are generated reactively rather than designed.

18.2 What to specifically test for AI-generated code

- [] **Authorization tests with multiple real accounts** for every object-access endpoint (Section 7.1) — this is the test category most often missing entirely from AI-generated test suites, because the model tests with a single happy-path user.
- [] **Adversarial/malformed-input tests**: oversized payloads, wrong types, missing required fields, unexpected extra fields, injection-pattern strings.
- [] **Regression tests for every fixed bug** — captured as a permanent test, not just patched and moved

on, so the same AI-generated mistake can't quietly resurface in a later regeneration of similar code.

- [] **Concurrency/race-condition tests** for anything involving shared mutable state, inventory counts, or payment processing.

18.3 Non-functional testing

- [] **Load/performance testing** against realistic (not optimistic) traffic patterns before launch — including a deliberate spike test, since "it handled my manual testing fine" tells you nothing about concurrent load.
- [] **Accessibility testing** integrated into the normal test suite, not a separate one-time audit (Section 24).
- [] **Security testing** (SAST/DAST/dependency scanning) runs as part of the standard pipeline, not as an occasional manual exercise (Section 15.5, Section 19).
- [] **Chaos/failure-injection testing** for critical paths where feasible — verify the system behaves correctly when a dependency times out or returns an error, not only when everything succeeds.

18.4 CI gating

- [] The full relevant test suite is a hard merge gate — not advisory, not skippable under time pressure,

and not something a human can override without an explicit, logged justification.

- [] Flaky tests are fixed or removed, not silently ignored/re-run-until-green — a culture of re-running failing tests until they pass quietly destroys the entire signal the test suite exists to provide.
-

19. CI/CD pipeline

19.1 Pipeline integrity

- [] Builds run on dedicated, hardened, ephemeral infrastructure — not a long-lived shared runner or a developer's own machine, where state (and compromise) can persist between builds.
- [] Pipeline configuration (and any infrastructure-as-code it depends on) is reviewed with the same rigor as application code — a misconfigured CI/CD pipeline is itself a direct production access vector.
- [] Branch protection requires passing checks and at least one human review before merge to the deployment branch — including for AI-agent-authored PRs, with no "trusted bot" bypass.
- [] Build inputs are pinned (dependency versions, base images, action/plugin versions by digest, not

floating tags) so a build is reproducible and not silently altered by an upstream change.

19.2 Required automated gates, in order

1. ☐ Lint / static analysis
2. ☐ Dependency & package-existence verification, including a hallucinated-package check (Section 4.1) and known-CVE scan
3. ☐ Secret scanning
4. ☐ Unit + integration test suite
5. ☐ SAST scan
6. ☐ Build artifact creation + SBOM generation
7. ☐ Container image scan (Section 20)
8. ☐ Deploy to staging + smoke/E2E tests
9. ☐ Manual or automated approval gate for production deploy

19.3 Deployment safety

- ☐ Deploys are progressive (canary/blue-green/rolling), not instantaneous full cutover — so a bad release affects a small slice of traffic before it affects everyone.
- ☐ Automated rollback is wired to your health checks/error-rate monitoring, not a manual scramble to remember the rollback command during an incident.

- [] Database migrations are decoupled from application deploys where possible (Section 8.3) so either can be rolled back independently.
- [] Feature flags gate risky changes so they can be disabled instantly without a redeploy.

19.4 Environment parity

- [] Staging mirrors production configuration (versions, scaling topology where feasible, security settings) closely enough that "it worked in staging" is meaningful evidence — a staging environment that's materially different from production gives false confidence.
 - [] Secrets and data differ appropriately between environments — production credentials and real customer data never flow into staging/dev environments with weaker access controls.
-

20. Docker & containers

- [] **Run as a non-root user inside the container** — a generated `Dockerfile` defaults to root unless explicitly told otherwise, which turns a contained compromise into a much larger one.
- [] **Use minimal base images** (distroless or slim variants) — a full OS image inside a container is

unnecessary attack surface and unnecessary CVEs to track.

- [] **Multi-stage builds** so build tools, source code, and intermediate artifacts never ship in the final runtime image.
- [] **No secrets baked into image layers** — credentials passed as build args are still recoverable from image history; use runtime injection (mounted secrets, environment from a secrets manager) instead.
- [] **Images are pinned to a specific digest**, not a floating tag like `latest`, and are scanned for known vulnerabilities as part of CI before being pushed/deployed.
- [] **Resource limits are set** (CPU/memory) for every container — an unbounded container is one runaway process away from taking down everything sharing its host/node.
- [] **Read-only root filesystem** where the application doesn't need to write to its own container filesystem, with explicit writable mounts only where actually required.
- [] **Health checks are defined** (`HEALTHCHECK` / orchestrator-level liveness & readiness probes) so the orchestrator can detect and replace an unhealthy container automatically.
- [] **Container registry access is locked down** — only authorized pipelines can push, and pulls in

production are restricted to the verified, signed images your pipeline produced, not an arbitrary public tag.

- [] **No privileged containers and no unnecessary host mounts** (especially the Docker socket) — both are common in quickly-generated `docker-compose` files and both grant a path to full host compromise if the container is ever compromised.
-

21. Cloud deployment & scalability

21.1 Infrastructure as code

- [] Infrastructure is defined as code (Terraform/CloudFormation/Pulumi/equivalent) and reviewed the same way application code is — not clicked together manually in a console, which is unrepeatable and undocumented.
- [] IaC changes go through the same plan-review-apply gating as application deploys, with a clear diff of what will actually change before it's applied.

21.2 Scalability design

- [] The application is stateless at the compute layer (session/cache state lives in a shared store,

not in-process) so it can scale horizontally without sticky-session hacks.

- [] Auto-scaling policies are configured and tested under realistic load — not left at default thresholds that were never validated against your actual traffic profile.
- [] Database connection pooling is configured deliberately; verify the pool size doesn't exceed what the database can actually sustain under full horizontal scale-out (a very common "worked at low scale, exhausted DB connections at high scale" failure).
- [] Stateful dependencies (database, cache, queue) have their own scaling/sharding plan independent of the application tier — application scaling alone doesn't help if the database is the bottleneck.
- [] CDN is in front of static assets and cacheable responses to reduce origin load and improve global latency.

21.3 Resilience

- [] Multi-AZ (and, where justified, multi-region) deployment for anything with a real availability requirement — a single point of failure in infrastructure undermines every other reliability investment.

- [] Timeouts, retries with backoff, and circuit breakers are configured on every call to a downstream dependency — generated service-to-service code very commonly omits a timeout entirely, which turns one slow dependency into a cascading outage.
- [] Graceful shutdown is implemented (drain in-flight requests, deregister from load balancer before the process exits) so deploys and scale-down events don't drop active requests.
- [] Dependencies are inventoried with an explicit understanding of what happens to your application when each one is degraded or unavailable.

21.4 Cost and capacity

- [] Cost monitoring/alerts are in place before launch, not discovered via a surprising bill — auto-scaling without cost guardrails is a common way an AI-assisted launch becomes an expensive one.
 - [] Capacity planning accounts for realistic growth and worst-case spikes (marketing launch, viral moment), not just current baseline traffic.
-

22. Backups & disaster recovery

- [] **The 3-2-1 rule, at minimum:** at least three copies of critical data, on at least two different storage media/systems, with at least one copy stored somewhere physically/logically separate (different region/account) from production — so a single compromised account or regional outage can't take out your data and every backup of it simultaneously.
- [] **Recovery Point Objective (RPO) and Recovery Time Objective (RTO) are explicitly defined** for each critical system — "how much data can we afford to lose" and "how long can we afford to be down" are business decisions, not engineering afterthoughts, and the backup strategy must be built to actually meet them.
- [] **Backups are automated and run on a schedule that matches your defined RPO** — not "someone remembers to run it."
- [] **Restores are actually tested on a schedule** — an untested backup is a hypothesis, not a recovery plan. Run a real restore-and-verify drill regularly, not only when a real incident forces you to find out whether it works.
- [] **Backups are immutable/protected from deletion** for at least a defined retention window (object lock / write-once semantics), specifically to survive a ransomware scenario where an

attacker with write access tries to delete backups along with production data.

- [] **Backup access is independently controlled** — the credentials that can delete/modify backups are not the same broadly-scoped credentials your application uses day to day.
 - [] A documented, rehearsed disaster recovery runbook exists — who declares an incident, who has authority to initiate a restore, and what the actual step-by-step recovery procedure is, written down before you need it under pressure.
-

Part IV — Reach and obligations

23. SEO (for web applications)

23.1 Core Web Vitals

Google's published "good" thresholds, measured as real-user field data at the 75th percentile:

Metric	Measures	"Good" threshold
LCP (Largest Contentful Paint)	Loading speed	Under 2.5 seconds
INP (Interaction to Next Paint)	Responsiveness to <i>every</i> interaction, not just the first	Under 200 milliseconds
CLS (Cumulative Layout Shift)	Visual stability	Under 0.1

All three must pass simultaneously at the 75th percentile for a page/site to be classified "Good" in Search Console — passing two out of three isn't sufficient. INP in particular has the lowest real-world pass rate of the three, because fixing it requires actually restructuring how JavaScript handles interactions (breaking up long main-thread tasks), not a quick config change like the other two.

- [] LCP: the largest above-the-fold element (usually a hero image or heading) is preloaded and never lazy-loaded; render-blocking CSS/JS is minimized; server response time (TTFB) is fast, backed by a CDN.

- [] INP: no long (>50ms) JavaScript tasks block the main thread during user interaction; heavy third-party scripts (chat widgets, ad tags, analytics) are deferred/lazy-loaded rather than blocking initial interactivity.
- [] CLS: every image, video, and embed has explicit dimensions or a reserved `aspect-ratio`; fonts use `font-display: swap` with matched fallback metrics; nothing (cookie banners, ads, dynamically-inserted content) shifts existing content after it has rendered.
- [] Measured with real-user field data (Search Console / CrUX), not lab data alone — lab tests on a fast machine on a fast connection routinely miss what real mobile users on real networks experience.

23.2 Technical SEO fundamentals

- [] Server-side rendering or static generation for anything that needs to be indexed — a pure client-side-rendered SPA that requires JavaScript execution to show content is a real (if shrinking) indexing risk; verify with an actual rendered-HTML check, not an assumption that "Google handles JS fine now."
- [] Every public page has a unique, accurate `<title>` and meta description — generated

pages/templates frequently ship with a single repeated title across every route.

- [] Semantic HTML and a logical heading hierarchy (`<h1>` once per page, nested correctly) – this also directly supports accessibility (Section 24).
- [] `robots.txt` and an XML sitemap exist, are accurate, and don't accidentally block pages you want indexed (a very common AI-generated `robots.txt` mistake: disallowing everything by default, copied from a staging-environment template).
- [] Canonical tags are set correctly to avoid duplicate-content issues across parameterized/paginated URLs.
- [] Structured data (schema.org/JSON-LD) is added for relevant content types (articles, products, FAQs) and validated – incorrect structured data can suppress rich results rather than earn them.
- [] Mobile-first: verify the *mobile* rendering and performance specifically, since indexing and ranking are primarily based on the mobile version of the page.
- [] HTTPS everywhere, with a single canonical domain (www vs. non-www, trailing slash consistency) – redirect everything else to it with a 301, not a 302.

- [] Broken links and redirect chains are checked before launch and periodically after — both degrade crawl efficiency and user experience.
-

24. Accessibility (WCAG)

24.1 What standard to build to

WCAG 2.2 at Level AA is the right default target for new applications in 2026 — it's the current W3C recommendation, it's a superset of WCAG 2.1 (so building to 2.2 also satisfies any requirement that cites 2.1), and it's the version most accessibility experts recommend even where the letter of a specific regulation still cites an older version. WCAG 3.0 is a future major revision still years from becoming a stable, legally-referenced standard — don't wait for it; build to 2.2 AA now. (Specific regulatory deadlines for your jurisdiction and sector should be confirmed directly, since they vary and continue to be updated — see Section 25.)

24.2 Verification checklist (the four WCAG principles — Perceivable, Operable, Understandable, Robust)

- [] Every meaningful image has accurate `alt` text; purely decorative images have empty `alt=""` so

screen readers skip them rather than announcing noise.

- [] **Color contrast** meets at least 4.5:1 for normal text and 3:1 for large text/UI components – verified with an actual contrast checker against your real design tokens, not eyeballed.
- [] **All functionality is operable by keyboard alone** – tab order is logical, nothing traps focus, and every interactive element (including custom-built components like modals, dropdowns, and date pickers, which generated code frequently gets wrong) is reachable and operable without a mouse.
- [] **Visible focus indicators** on every interactive element – never `outline: none` without a clearly visible replacement, which is a default a generated CSS reset will often silently include.
- [] **Touch targets** meet the WCAG 2.2 minimum target size guidance, and forms support a documented accessible-authentication path (e.g., allow password managers/paste into password fields, and don't require a cognitive test like solving a puzzle as the *only* path to authenticate) – both are new criteria added specifically in 2.2.
- [] **Semantic HTML and ARIA used correctly** – native elements (`<button>` , `<nav>` , `<label>`) are preferred over `<div>` + ARIA role reconstructions; where ARIA is necessary,

roles/states are kept in sync with actual UI state (e.g., `aria-expanded` genuinely reflects whether something is open).

- **[] Forms have programmatically associated labels**, clear error identification, and accessible error messages that are announced to assistive technology, not just visually styled red text.
- **[] Captions/transcripts** for video and audio content.
- **[] Page structure is navigable**: proper heading hierarchy, skip-to-content link, and landmark regions (`<header>` , `<main>` , `<nav>` , `<footer>`).
- **[] Tested with actual assistive technology** — at minimum a screen reader (VoiceOver/NVDA) pass on critical flows — not only automated scanner output. Automated tools reliably catch maybe a third of real accessibility issues; the rest require a human (ideally one who uses assistive technology) actually navigating the flow.
- **[] Accessibility overlay widgets are not a substitute for fixing the underlying markup** — regulators and courts have explicitly rejected "we installed a widget" as sufficient remediation; fix the code.

24.3 Process

- **[] Accessibility checks run automatically in CI** (axe-core or equivalent) on every build, catching

regressions before merge — not as a one-time pre-launch audit that's never revisited.

- [] A documented accessibility statement / VPAT exists if you sell into enterprise, government, or education customers, who will frequently require one during procurement.
-

25. Legal & regulatory compliance

Compliance requirements are jurisdiction- and sector-specific and change frequently — treat everything below as the checklist of *areas to verify with current sources or counsel*, not a substitute for that verification. The regulatory landscape (especially in the US, where state-level privacy laws have proliferated and California has substantially expanded its enforcement program) has changed materially even within the last year, so don't rely on prior knowledge of "what GDPR/CCPA require" without re-checking current requirements.

25.1 Data privacy

- [] **Know which regimes actually apply to you** based on whose data you process and where, not just where your company is incorporated — GDPR applies based on whose data is processed (EU residents), not where the processing company is

based; an increasing number of US states now have their own comprehensive privacy laws with their own specific notice, opt-out, and assessment requirements.

- **[] Data inventory exists:** what personal data you collect, why, where it's stored, who it's shared with, and how long it's retained — you cannot honestly answer a user's access/deletion request, or a regulator's audit, without this.
- **[] Consent/notice mechanics match the regime:** GDPR-style jurisdictions generally require opt-in consent before non-essential tracking/cookies fire; CCPA-style US state laws generally use an opt-out model (data can flow by default, but users must have a working, honored mechanism to opt out of sale/sharing) — these are different mechanics, not interchangeable, and a site serving both audiences needs geo-aware logic rather than one global banner.
- **[] User rights are actually implemented, not just promised in a privacy policy:** access, correction, deletion, and export requests have a real working workflow and a defined response timeframe — test the deletion flow end-to-end, including in backups and analytics pipelines, not just the primary database.
- **[] Breach notification process exists and meets the relevant timelines** (commonly as fast as 72

hours under GDPR-style regimes) – defined *before* an incident, including who decides, who notifies whom, and the template for doing so.

- ☐ **Data processing agreements** are in place with every third-party vendor/subprocessor that touches personal data on your behalf.
- ☐ **Privacy-by-design basics**: collect only what you actually need (data minimization), and don't repurpose data collected for one stated purpose for an unrelated one without fresh consent/notice.
- ☐ If your product makes automated decisions with significant effects on users (credit, employment-adjacent, eligibility scoring, profiling) or trains models on user data, check whether that specifically triggers additional assessment/disclosure obligations in the regimes that apply to you – this is an area of active regulatory expansion.

25.2 Accessibility law

- ☐ Confirmed current obligations for your specific sector and jurisdiction – requirements (which WCAG version, which deadline, which entities are covered) differ between, for example, US public-sector ADA Title II obligations, US private-sector Title III litigation exposure, and EU European Accessibility Act/EN 301 549 obligations, and deadlines in this area have been actively shifting.

Building to WCAG 2.2 AA (Section 24) covers you against essentially every current and pending version of these requirements regardless of which specific one technically applies.

25.3 Sector-specific regimes

- [] **Payment data:** PCI DSS scope assessed honestly — the simplest way to reduce obligation is to never let your own servers touch raw card data (tokenize via a PCI-compliant processor) rather than build card handling yourself.
- [] **Health data:** HIPAA (US) or equivalent applies based on the *type* of data and *who* is handling it, not just "we're a health app" — verify whether you're actually a covered entity/business associate, and what that means for storage, access logging, and breach response specifically.
- [] **Children's data:** COPPA (US) and equivalent regimes elsewhere impose materially stricter consent and data-handling rules if your service is directed at or knowingly used by children — verify before assuming a general-audience privacy policy is sufficient.
- [] **AI-specific regulation:** if your product uses AI in ways that make automated or significant decisions about people, check emerging AI-specific regulatory obligations (transparency, risk classification, human oversight requirements) that

are actively coming into force in multiple jurisdictions — this is a genuinely moving target; verify current status rather than relying on older assumptions.

25.4 Terms, licensing, and IP

- [] Terms of Service and Privacy Policy are reviewed by someone qualified to do so, not generated wholesale and shipped unread — they are the legal contract governing your relationship with every user.
- [] Every open-source dependency's license is checked for compatibility with your intended use (especially commercial/closed-source distribution) — AI-suggested dependencies are added for functionality, not for license fit, and copyleft obligations can be a real constraint you don't want to discover after shipping.
- [] Generated content (AI-written copy, AI-generated images) used in the product is checked against your jurisdiction's current understanding of AI-output copyright/ownership status and against any third-party rights the training data might implicate, particularly for anything closely resembling a specific real, identifiable work or person.

The standing instruction for this entire section: none of the above is legal advice, and compliance

requirements change — confirm current obligations with qualified counsel or current official regulatory guidance before launch, especially for anything involving regulated data (health, payments, children) or multi-jurisdiction reach.

Part V — Shipping

26. The final production launch checklist

This is the go/no-go gate. Every item should have a named owner who has actually verified it — not a box checked because "that's probably fine."

Code & review

- [] Every AI-generated diff in the release has gone through the Stage 1–4 review workflow in Section 3 — no exceptions for "it was a small change."
- [] Every dependency added during this development cycle has been verified to actually exist and been vetted (Section 4.1, 15.3).
- [] No secrets, debug endpoints, or `console.log / print`-style debug output remain in the shipped code.

Security

- ☐ Every endpoint that accepts an object ID has a passing two-account authorization test (Section 7.1).
- ☐ OWASP Top 10:2025 categories have been walked deliberately against this release, not assumed covered (Section 15.1).
- ☐ Automated SAST, dependency, and secret scans are green on the release build.
- ☐ A real human security review (internal or external pen test) has happened for this release if it touches auth, payments, or sensitive data.
- ☐ Security headers, CORS, and CSRF protections are verified in the actual deployed environment, not just in local dev config.

Data

- ☐ Backups are running, tested with an actual restore, and meet the defined RPO/RTO (Section 22).
- ☐ Migrations have been reviewed for backward compatibility and lock behavior at production scale (Section 8.3).
- ☐ Sensitive data is encrypted at rest and in transit; access is least-privilege.

Reliability

- ☐ Load testing has been run against realistic and spike traffic patterns, and the system behaved acceptably (Section 18.3, 21.2).
- ☐ Timeouts, retries, and circuit breakers exist on every external dependency call (Section 21.3).
- ☐ Health checks, auto-scaling, and rollback are configured and have been tested, not just configured (Section 19.3, 20).
- ☐ On-call rotation, alert routing, and runbooks exist and are known to the people on them (Section 17).

User-facing quality

- ☐ Loading, empty, error, and partial-failure states are designed and verified for every critical screen (Section 5.1).
- ☐ Core flows tested on real mobile devices and the actual supported browser matrix (Section 5.3, 6.4).
- ☐ Accessibility: automated scan is clean and a manual keyboard + screen-reader pass has been done on critical flows (Section 24).
- ☐ Core Web Vitals measured (not assumed) and within target thresholds on real-world conditions (Section 23.1).

Compliance & legal

- ☐ Privacy policy and Terms of Service are current, reviewed, and actually match what the product does.
- ☐ Data subject rights (access/delete/export) work end-to-end, including in backups and analytics.
- ☐ Sector-specific obligations (payments, health, children's data, accessibility law) have been explicitly checked against current requirements, not assumed unchanged from a prior project (Section 25).

Observability

- ☐ Structured logging, metrics, and tracing are live in production for this release before it receives real traffic, not added reactively after the first incident.
- ☐ Dashboards exist for both system health and the specific business metrics this release is meant to move.

Rollback readiness

- ☐ A specific, tested rollback procedure exists for this release (not "we'll figure it out") and the team knows who can pull the trigger.
- ☐ Feature flags are in place for the riskiest parts of the release so they can be disabled without a full redeploy.

If any item above is unverified, the answer is "not ready," regardless of deadline pressure. The entire premise of this handbook is that AI-generated code earns trust through verification, not through how confident it looks or how fast it got you here.

Appendix A — Condensed master checklist

A printable, one-pass summary. Each line corresponds to a fully detailed section above.

- ☐ Spec-style prompts used for every non-trivial generation; agent tasks explicitly scoped (§2)
- ☐ Every AI diff passes automated gates, then a human review with the AI-specific lens (§3)
- ☐ No hallucinated/unverified packages; SBOM generated; fabricated APIs checked against real docs (§4)
- ☐ UI has loading/empty/error/partial-failure states; destructive actions confirm; mobile verified on real devices (§5)
- ☐ No secrets in client bundle; XSS-safe rendering; CSP set; CWV-aware image/JS handling (§6)
- ☐ Two-account authorization test on every object-access endpoint; mass-assignment blocked; SSRF

guarded; idempotency on retryable writes (§7)

- [] Real FK/constraints enforced in DB; no string-built SQL; indexes verified via query plan; safe migrations (§8)
- [] Modern password hashing or passwordless; short-lived tokens with real revocation; verified JWT claims (§9)
- [] Centralized, default-deny, server-side authorization on every request; least-privilege service credentials (§10)
- [] Server-side schema validation on every input; structure and bounds checked before business logic runs (§11)
- [] Rate limits on every public endpoint, tiered for auth/sensitive flows, enforced server-side with shared state (§12)
- [] Explicit cache invalidation; no shared cache of personalized data; stampede protection (§13)
- [] Uploads verified by content not extension; storage private-by-default; short-lived, server-controlled presigned URLs (§14)
- [] OWASP Top 10:2025 walked deliberately; secrets in a vault; supply chain (SBOM, pinned deps, scanned) verified (§15)
- [] Structured logs; security events logged with context; no secrets/PII in logs; correlated trace IDs (§16)

- [] Metrics + logs + traces live; alerts tied to user impact with a tested runbook; synthetic checks on critical flows (§17)
- [] Pyramid-shaped test suite including authorization and adversarial-input tests; CI gate is non-bypassable (§18)
- [] Hardened ephemeral build infra; full automated gate sequence; progressive deploys with automated rollback (§19)
- [] Non-root, minimal, pinned, scanned container images; resource limits; no privileged containers (§20)
- [] IaC reviewed like code; stateless compute; pooled/sharded data layer; timeouts+retries+circuit breakers everywhere (§21)
- [] 3-2-1 backups meeting a defined RPO/RTO; restores actually tested; backups immutable and separately credentialed (§22)
- [] Core Web Vitals measured and passing on real-user data; technical SEO fundamentals in place (§23)
- [] WCAG 2.2 AA verified with automated scan + real keyboard/screen-reader pass, not an overlay widget (§24)
- [] Current privacy, accessibility, and sector-specific legal obligations verified against current

sources, not assumed (§25)

- [] Full launch checklist signed off by named owners per area, with a tested rollback plan (§26)

Appendix B — Recommended tooling by category

Tool landscapes shift quickly — verify current options and pricing before standardizing, but these categories are the right shape to fill regardless of which specific vendor you pick:

- **Static analysis / SAST:** Semgrep, CodeQL, SonarQube/SonarLint
- **AI-specific PR review:** tools that diff-review AI-generated code for security and logic issues, layered on top of (not instead of) human review
- **Dependency / package verification & SCA:** Snyk, Socket.dev, OWASP Dependency-Check, npm/PyPI registry-verification CLIs
- **Secret scanning:** gitleaks-class pre-commit and CI scanners, plus your cloud provider's native secret-detection service
- **Secrets management:** HashiCorp Vault, AWS Secrets Manager, GCP Secret Manager, Azure Key Vault

- **Auth/identity:** a managed identity provider (Auth0/WorkOS/Clerk/Keycloak/Authentik-class) or framework-native auth (e.g., Authlib/NextAuth-class libraries) plus WebAuthn/passkey support
 - **Authorization/policy engines:** a centralized policy library (OPA/Cerbos-class) once RBAC outgrows simple role checks
 - **Container/image scanning:** Trivy, Gype, your registry's native scanning
 - **CI/CD:** GitHub Actions / GitLab CI / equivalent, with SLSA-aligned provenance generation
 - **Observability:** an OpenTelemetry-based stack (traces/metrics/logs), paired with an APM and a paging/alerting tool
 - **Accessibility testing:** axe-core (automated, in CI) plus manual screen-reader passes
 - **Performance/Core Web Vitals:** Google PageSpeed Insights / Search Console field data, Lighthouse for lab data, a real-user-monitoring tool for ongoing tracking
 - **Backup/DR:** your cloud provider's native backup service with cross-region/cross-account replication, or a dedicated backup platform with immutability/object-lock support
-

This handbook is a verification framework, not a substitute for domain expertise, legal counsel, or a security professional's review of your specific system. Treat every checklist item as a starting point for genuine verification, not a box to check because the relevant technology exists somewhere in your stack.